

역기구학 2일차 과제3 보고서, 김도훈, 2026406041

<목차>

1. Numerical IK란?
2. 자코비안 행렬이란?
3. 자코비안 행렬을 사용한 Numerical IK
4. singularity
5. singularity 예외 처리

1. Numerical IK란?

Numerical IK란 수치적 역기구학이다. 로봇의 목표 위치를 만족하는 각 관절의 각도를 반복 연산과 근사치를 통해 해를 찾아내는 방법이다. 이 수치적 역기구학을 사용하는 이유는 해석적 역기구학으로는 실제로 손목이 구형이 아닌 협업 로봇, 7-DOF 이상의 로봇팔에서는 해가 무한히 많거나 찾지 못하는 경우가 있다. 결과적으로 정확히 로봇의 End-effector의 위치를 찾기 어렵다. 그래서 제안된 것이 수치적 역기구학이다. 위에서 설명한대로 해석적 역기구학으로 풀 수 없는 것들은 수치적 역기구학을 사용해 반복적으로 또한 수치적 역기구학을 푼다를 근사한다. 또한 수치적 역기구학을 푼다면 IK를 직접 풀지 않아도 로봇을 원하는 위치로 유도할 수 있다. ¹⁾

2. 자코비안 행렬(Jacobian matrix)

이 수치적 역기구학 방법으로 자코비안(Jacobian) 행렬을 이용한 방법이 많이 사용된다. 그러므로 자코비안 행렬에 대한 먼저 숙지를 하고 있어야한다.

자코비안 행렬이란 관절 속도 벡터와 말단(end-effector) 속도 벡터 사이의 선형 관계를 나타내는 행렬이다. 관절이 움직이는 속도가 end-effector의 속도로 어떻게 변환되는지 연결해 준다. ²⁾ 관절각 θ_1, θ_2 와 링크 길이 L_1, L_2 인 2차원 로봇팔을 예시로 자코비안을 설명해보겠다. 일단 로봇팔의 end-effector 위치는

$$x = L_1 \cos \theta_1 + L_2 \cos(\theta_1 + \theta_2), y = L_1 \sin \theta_1 + L_2 \sin(\theta_1 + \theta_2)$$

이다. 그리고 end-effector의 속도는 $\dot{x}, \dot{y}$, 각 관절의 속도 $\dot{\theta}_1, \dot{\theta}_2$ 로 표현하겠다. end-effector의 속도를 각 관절의 속도와 자코비안 행렬로 표현하면 다음과 같다.

$$\begin{bmatrix} \dot{x} \\ \dot{y} \end{bmatrix} = J(\theta_1, \theta_2) \begin{bmatrix} \dot{\theta}_1 \\ \dot{\theta}_2 \end{bmatrix}$$

1) 역기구학 - 수치적 풀이 (<https://jhpark0518.tistory.com/132>)

2) 자코비안 행렬 (<https://jhpark0518.tistory.com/4>)

J는 자코비안 행렬이다. 위 식에서 알 수 있듯이 자코비안 행렬은 각 위치를 관절 각도에 대해 편미분한 것이다. 자코비안 행렬은 다음과 같이 정의된다.

$$J = \begin{bmatrix} \frac{\partial x}{\partial \theta_1} & \frac{\partial x}{\partial \theta_2} \\ \frac{\partial y}{\partial \theta_1} & \frac{\partial y}{\partial \theta_2} \end{bmatrix} \quad J = \begin{bmatrix} -L_1 \sin \theta_1 - L_2 \sin(\theta_1 + \theta_2) & -L_2 \sin(\theta_1 + \theta_2) \\ L_1 \cos \theta_1 + L_2 \cos(\theta_1 + \theta_2) & L_2 \cos(\theta_1 + \theta_2) \end{bmatrix}$$

실제로 end-effector의 위치를 사용해 계산하면 오른쪽 식과 같다. 오른쪽 식에 각 관절의 속도를 곱하면 end-effector의 속도가 다음과 같이 나온다.

$$\begin{bmatrix} \dot{x} \\ \dot{y} \end{bmatrix} = J \begin{bmatrix} \dot{\theta}_1 \\ \dot{\theta}_2 \end{bmatrix} = \begin{bmatrix} (-L_1 \sin \theta_1 - L_2 \sin(\theta_1 + \theta_2)) \dot{\theta}_1 + (-L_2 \sin(\theta_1 + \theta_2)) \dot{\theta}_2 \\ (L_1 \cos \theta_1 + L_2 \cos(\theta_1 + \theta_2)) \dot{\theta}_1 + (L_2 \cos(\theta_1 + \theta_2)) \dot{\theta}_2 \end{bmatrix}$$

이렇게 자코비안 행렬을 사용하면 각도를 변화시켰을 때 end-effector의 위치가 얼마나 바뀌었는지 알 수 있다. 자코비안 행렬의 정의에서 1열이 의미하는 것은 θ_1 , 1번 관절을 움직였을 때 end-effector의 위치가 어떻게 움직였는지를 의미한다. 두 번째 열은 2번 관절로 생각해야 한다. 즉 자코비안 행렬은 각 관절의 움직임이 end-effector 움직임에 미치는 영향을 모아 놓은 행렬이라고 생각하면 이해하기 쉽다.

3. 자코비안 행렬을 사용한 Numerical IK

위의 설명은 관절각의 변화를 자코비안 행렬을 사용해 end-effector의 위치를 구했다. 하지만 Numerical IK에서는 end-effector의 위치를 알고 있을 때 자코비안 행렬을 사용해 관절각을 구해야 한다.

$$\begin{bmatrix} \dot{x} \\ \dot{y} \end{bmatrix} = J \left(\begin{bmatrix} \dot{x} \\ \dot{y} \end{bmatrix} \right) \begin{bmatrix} \dot{\theta}_1 \\ \dot{\theta}_2 \end{bmatrix} \rightarrow \text{반대 과정} \rightarrow \begin{bmatrix} \dot{\theta}_1 \\ \dot{\theta}_2 \end{bmatrix} \approx \begin{bmatrix} \dot{x} \\ \dot{y} \end{bmatrix} J^{-1} \left(\begin{bmatrix} \dot{x} \\ \dot{y} \end{bmatrix} \right)$$

Numerical IK의 과정이 위에 보이는 반대 과정의 식과 같다. 목표 end-effector와 현재 end-effector의 위치 오차가 충분히 작아질 때까지 계속 반복하면 근사한 해를 찾을 수 있다. 이 오차가 충분히 작아질 때까지 반복해야 하기 때문에 한 번에 풀리진 않는다. 이 방법은 Newton-Raphson 방법의 IK 버전이다. Numerical IK 과정을 좀 더 자세히 설명하자면

1. 현재 관절 상태에서 end-effector의 위치를 구한다. 이 과정은 FK이다.

$$\begin{bmatrix} \dot{x} \\ \dot{y} \end{bmatrix} = \text{end effector}$$

2. 1번 과정에서 현재 end-effector의 위치를 구했다면 목표 end-effector의 위치와의 오차를 구한다.

$$\begin{bmatrix} \dot{x}_{\text{목표}} \\ \dot{y}_{\text{목표}} \end{bmatrix} - \begin{bmatrix} \dot{x} \\ \dot{y} \end{bmatrix} = \text{오차} \rightarrow \text{end effector가 이동해야 하는 거리}$$

3. 현재 관절의 상태에서 자코비안 값을 구한다. 자코비안 값을 구하는 이유는 움직여야 하

는 관절을 찾는다고 하더라도 현재 관절 상태에 따라 움직임이 달라질 수 있기 때문에 자코비안 값을 구하는 것이다. 이 과정은 매우 필수적이다.

$$\text{현재 자코비안 값} = J \left(\begin{bmatrix} \dot{x} \\ \dot{y} \end{bmatrix} \right)$$

4. $\begin{bmatrix} \dot{\theta}_1 \\ \dot{\theta}_2 \end{bmatrix} \approx \begin{bmatrix} \dot{x} \\ \dot{y} \end{bmatrix} J^{-1} \left(\begin{bmatrix} \dot{x} \\ \dot{y} \end{bmatrix} \right)$ 위의 식을 통해 관절각을 구한다.
5. 구한 관절각을 현재 관절각에 더하여 관절각을 업데이트 해준다.
6. 2번의 오차가 충분히 작아질 때까지 반복한다.

위의 과정이 자코비안 행렬을 사용하는 Numerical IK의 과정이다.

4. singularity

singularity란 특이점이다. 특이점은 로봇이 잘 움직이지 못하는 지점으로 쉽게 설명할 수 있다. 좀 더 자세히 설명하면 우리가 원하는 end-effector의 위치를 만들기 위해 관절각을 계산할 수 없는 지점이다. 즉 관절각이 무한히 변하거나 해를 결정할 수 없는 상황이 된다. 2차원 2관절 로봇팔을 예시로 들면 2차원 자코비안은

$$J = \begin{bmatrix} \frac{\partial x}{\partial \theta_1} & \frac{\partial x}{\partial \theta_2} \\ \frac{\partial y}{\partial \theta_1} & \frac{\partial y}{\partial \theta_2} \end{bmatrix} \quad J = \begin{bmatrix} -L_1 \sin \theta_1 - L_2 \sin(\theta_1 + \theta_2) & -L_2 \sin(\theta_1 + \theta_2) \\ L_1 \cos \theta_1 + L_2 \cos(\theta_1 + \theta_2) & L_2 \cos(\theta_1 + \theta_2) \end{bmatrix}$$

이고 자코비안 행렬의 역행렬이 존재하지 않으면 singularity가 발생한다. 위 자코비안 행렬의 determinant를 계산해보면 $\det(J) = L_1 L_2 \sin \theta_2$ 이고 이것이 0일 때 자코비안 행렬이 역행렬이 존재하지 않는 것이다. 이때 singularity가 발생한다. 2관절 로봇은 $\theta_2 = 0, \pi$ 일 때 singularity가 발생한다. 이것은 로봇팔이 완전히 펴졌거나 완전히 접힌 상태일 때를 의미한다.

이 singularity가 발생하면 Numerical IK과정에서 일단 계산이 불가능하다. 실제로는 아예 det값이 0이 아니라 0에 가까운 값이면 역행렬 값이 매우 커질 수 있다. 그래서 end-effector를 조금만 움직이고 싶는데 이것에 의해 관절각이 비정상적으로 크게 계산되어 엄청 큰 움직임이 생길 수 있다.

5. singularity 예외처리

Numerical IK의 singularity 처리에서 대표적인 방법이 Damped Least Squares(DLS)이다. 일반적으로

$$J^T(JJ^T + \lambda^2 I)^{-1} \Delta X$$

형태를 사용한다. 핵심은 JJ^T 이다. 이것이 singularity 때문에 문제가 생기더라도 $JJ^T + \lambda^2 I$ 처럼 작은 값을 더해줌으로써 위에서 설명한 관절각이 비정상적으로 크게 계산 되는 것을 막을 수 있다.³⁾

3) 역기구학 수치적 풀이 <https://jhpark0518.tistory.com/132>